Implicit Animation

Flutter – Chủ đề 8

Trương Nhật Hoàng

Hai loại animation trong Flutter

	Implicit (ngầm)	**Explicit (tường minh)**
Dev điều khiển	Giá trị đích	Dòng thời gian
Kiểu chuyển động	Một chiều A $\rightarrow$ B	Lặp, nhiều bước, theo gesture
`AnimationController`	Không cần	Bắt buộc
`dispose()`	Không cần	Bắt buộc
Widget điển hình	`AnimatedContainer`	`...Transition`, `AnimatedBuilder`

Nguyên tắc chọn: dùng implicit nếu đủ, chỉ chuyển sang explicit khi cần điều khiển dòng thời gian.

1. Implicit animation: hoạt ảnh ngầm định

- `duration`: thời gian chuyển động.
- Giá trị thuộc tính: **giá trị đầu** là giá trị hiện tại, **giá trị đích** là giá trị mới gán qua `setState`.
- `curve`: kiểu gia tốc, mặc định `Curves.linear` (chuyển động đều).

```
GestureDetector(
  onTap: () => setState(() => _expanded = !_expanded),  // doi gia tri dich
  child: AnimatedContainer(
    duration: Duration(milliseconds: 300),
    curve: Curves.easeOut,
    height: _expanded ? 160 : 80,
    decoration: BoxDecoration(
      color: _expanded ? Colors.blue.shade100 : Colors.grey.shade200,
      borderRadius: BorderRadius.circular(_expanded ? 24 : 8),
    ),
  ),
)
```

1. Implicit animation:

Dùng khi

- Chuyển động một chiều từ trạng thái A sang trạng thái B.
- Không lặp, không cần lệnh dừng hay đảo chiều.
- Đổi đích giữa chừng: tự chạy tiếp từ vị trí hiện tại sang đích mới.

Lợi ích

- Không cần `AnimationController`, không cần `dispose()`.
- Code ngắn: đổi widget tĩnh thành widget động tương ứng, thêm `duration`.

1.1. Quy ước tiền tố Animated

Widget tĩnh	Widget động
`Container`	`AnimatedContainer`
`Opacity`	`AnimatedOpacity`
`Padding`	`AnimatedPadding`
`Align`	`AnimatedAlign`
`Positioned`	`AnimatedPositioned`

- Animate nhiều thuộc tính cùng lúc: `AnimatedContainer`.
- Ngoại lệ cần nhớ:
  - `AnimatedSwitcher`: thay widget con, cần `key` khác nhau.
  - `AnimatedBuilder`: là **explicit**, cần controller.

```
AnimatedOpacity(
  duration: Duration(milliseconds: 200),
  opacity: _visible ? 1.0 : 0.0,
  child: Text('Hello'),
)
```

1.2. TweenAnimationBuilder

- Dùng khi thuộc tính cần animate không có widget `Animated...` sẵn (bộ lọc màu, đồng hồ số).
- `tween`: khoảng giá trị `begin` $\rightarrow$ `end`.
- `builder`: build lại giao diện mỗi khi giá trị nội suy thay đổi.
- Đổi `end` sau đó: chạy từ giá trị hiện tại sang `end` mới.

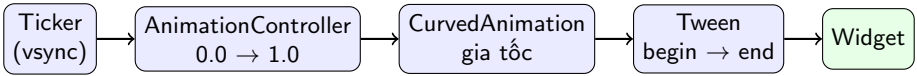
```
// Dong ho dem nguoc 10 -> 0
TweenAnimationBuilder<int>(
  tween: IntTween(begin: 10, end: 0),
  duration: Duration(seconds: 10),
  builder: (context, value, child) =>
      Text('$value'),
)
```

```
// Loc mau anh: trang -> xanh
TweenAnimationBuilder<Color?>(
  tween: ColorTween(begin: Colors.white,
      end: _night ? Colors.blue : Colors.white)
  duration: Duration(milliseconds: 500),
  child: Image.asset('photo.png'),
  builder: (context, color, child) =>
      ColorFiltered(
        colorFilter: ColorFilter.mode(
            color!, BlendMode.modulate),
        child: child),
)
```

2. Explicit animation: hoạt ảnh tường minh

Nhu cầu	**Ví dụ trong** `main.dart`
Animate vô hạn	Icon xoay: `_spin.repeat()`
Chuỗi nhiều bước, so le	Nảy rồi về: `forward().then(reverse)`
Kích hoạt, điều khiển bằng gesture	Kéo khối: `_drag.value += ...`

Ba thành phần chính:



2.1 AnimationController

```dart
// main.dart
class _DemoPageState extends State<DemoPage>
    with TickerProviderStateMixin {
  late final AnimationController _spin =
      AnimationController(
        vsync: this,
        duration: Duration(seconds: 1));

  @override
  void dispose() {
    _spin.dispose();    // dung ticker
    super.dispose();
  }
}
```

- Quản lý dòng thời gian, sinh `value` từ 0.0 đến 1.0 trong suốt `duration`.
- **Ticker**: gọi callback mỗi khung hình màn hình.
- **vsync**: nơi cấp Ticker cho controller, ở đây là State (nhờ mixin).
- Bắt buộc `dispose()`: dừng Ticker, tránh memory leak.
- Lệnh: `forward`, `reverse`, `repeat`, `stop`, `animateTo`.

2.2 Tween

- Controller chỉ sinh 0.0 → 1.0; `Tween` ánh xạ sang khoảng giá trị mong muốn.
- Kết nối: `tween.animate(controller)` trả về `Animation<T>`.
- Có sẵn: `Tween<double>`, `ColorTween`, `IntTween`, `Tween<Offset>`, ...

```
late final AnimationController _c =
    AnimationController(vsync: this, duration: Duration(seconds: 1));

// 0.0 -> 1.0  duoc anh xa thanh  50 -> 200
late final Animation<double> _size =
    Tween<double>(begin: 50, end: 200).animate(_c);

AnimatedBuilder(
  animation: _size,
  builder: (context, child) =>
      Container(width: _size.value, height: _size.value, color: Colors.teal),
)
```

2.3 CurvedAnimation

- Biến tiến trình tuyến tính của controller thành chuyển động có gia tốc.
- Thứ tự: controller → `CurvedAnimation` → `Tween`.
- Curve thường dùng: `easeIn`, `easeOut`, `easeInOut`, `elasticOut`, `bounceOut`.

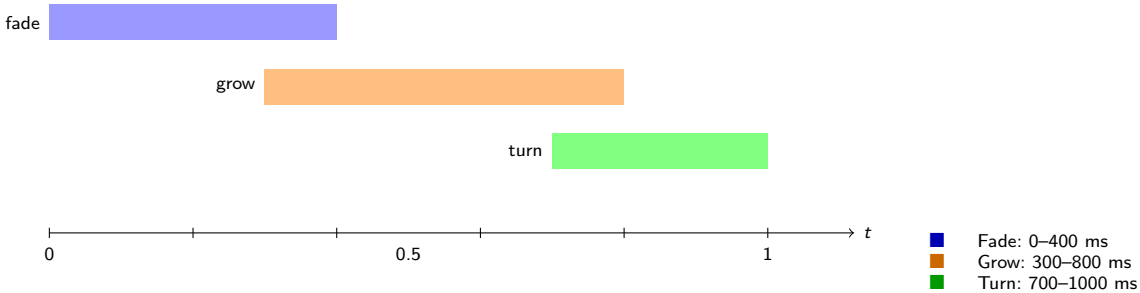
```
late final Animation<double> _size =
    Tween<double>(begin: 50, end: 200).animate(
      CurvedAnimation(parent: _c, curve: Curves.easeOut),
    );

// main.dart: nut tim - 1.0 -> 1.4 -> 1.0 (dung lowerBound/upperBound thay Tween)
late final AnimationController _heart = AnimationController(
    vsync: this, lowerBound: 1.0, upperBound: 1.4);

ScaleTransition(scale: _heart, child: Icon(Icons.favorite))
```

2.4 Staggered animation: hoạt ảnh so le nhiều bước

- Một `AnimationController` điều phối nhiều animation.
- `Interval(start, end)` xác định khoảng thời gian hoạt động của từng animation (theo tỉ lệ `duration`).
- Các animation dùng chung controller nên được đồng bộ trên cùng một dòng thời gian.



Nguyên tắc quản lý hiệu năng

1. Giảm rebuild: không `setState` mỗi khung hình, dùng `...Transition` / `AnimatedBuilder`.
2. Tận dụng tham số `child` cho phần tĩnh.
3. Ưu tiên `Transform` / `FadeTransition` hơn thay đổi bố cục.
4. Dùng `const`, không tạo đối tượng trong `build()`.
5. Giải phóng tài nguyên theo vòng đời State (`dispose()`, `cancel()`).
6. Không chặn UI thread: I/O dùng `Future`, tính toán nặng dùng `Isolate`.
7. Đo lường ở profile mode bằng Flutter DevTools.

3.1 Gesture: tín hiệu đầu vào của tương tác

`GestureDetector` chuyển sự kiện chạm thô thành 4 nhóm cử chỉ, mỗi nhóm có callback và tham số riêng:

Nhóm	**Callback**	**Tham số chính**	**Ứng dụng**
Chạm	`onTap`, `onDoubleTap`, `onLongPress`	—	Kích hoạt, chọn mục, mở menu
Kéo / trượt	`onPanUpdate` (hoặc `...DragUpdate` theo trục)	`delta.dx`, `delta.dy`	Cộng dồn vào vị trí + `setState()` → bám tay 1:1
Vuốt nhanh	`onPanEnd`	`velocity.pixelsPerSecond`	Vượt ngưỡng (vd. > 1000 px/s) → xoá / chuyển trang
Thu phóng / xoay	`onScaleUpdate`	`scale`, `rotation`	Bản đồ, gallery ảnh

Gesture là **input** $\Rightarrow$ khi nào cần **Animation phản hồi**?

3.2 Thời điểm kích hoạt Animation

Animation thường được dùng ở 3 thời điểm:

1. **Phản hồi tức thì:** vừa chạm / kéo thì nút co nhẹ, thẻ trượt theo $\rightarrow$ xác nhận hệ thống đã nhận thao tác.
2. **Chuyển đổi trạng thái:** menu trượt ra từ lề, danh sách $\leftrightarrow$ lưới $\rightarrow$ mắt theo dõi được quỹ đạo của widget.
3. **Thu hút chú ý:** thông báo quan trọng, hoàn tất tiến trình (icon tim nảy, nút rung nhẹ).

Gesture đúng lúc + Animation đúng thời điểm $\Rightarrow$ 3 tác động UX

3.3 Tác động tới trải nghiệm người dùng (UX)

1. **Timing:** phản hồi ngắn nên nằm trong **200–300 ms**; < 100 ms khó nhận ra, > 500 ms gây cảm giác trì trệ.
2. **Giảm tải nhận thức:** chuyển động mượt giúp người dùng tự nối trạng thái cũ với trạng thái mới, không bị giật.
3. **Ngăn thao tác nhầm:** kết hợp gesture với ngưỡng; chưa vượt ngưỡng thì *snap back*, vượt mới thực hiện.

Tổng kết

- **Implicit**: chỉ định `duration`, `curve` và giá trị đích; tiền tố `Animated`; `TweenAnimationBuilder` cho thuộc tính tuỳ ý.
- **Explicit**: dùng khi cần vô hạn, nhiều bước hoặc gesture.
- `AnimationController` (Ticker, vsync, `dispose`) → `CurvedAnimation` → `Tween`.
- **Staggered**: một controller, nhiều `Interval`, đồng bộ từng khung hình.
- **Hiệu năng**: ít rebuild (`...Transition`, `child`, `const`), ưu tiên Transform, `dispose()` đúng vòng đời, không chặn UI thread, đo ở profile mode.